AIrcraft

Wilke Grosche

October 12, 2026

Abstract

This work addresses the problem of minimum time path planning for models with nonlinear surrogate dynamics, in particular those in which the dynamics are in part given by black-box models.

1 Introduction

1.1 Project Overview

This project is part of a greater effort to work towards shape-optimal control policies. Here we set up a framework for

1.2 Problem Description

Concretely we have an unmanned fixed wing aircraft (hereafter referred to as a glider) that we want to fly between two planes as frequently as possible within a fixed time frame. A visual representation is given in fig. 1. To solve this problem we need to formulate it a bit more abstractly.

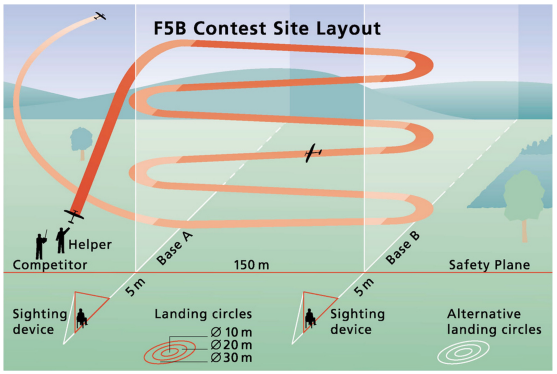


Figure 1: Problem setting for the F5B RC fixed wing aircraft competition.[2]

2 Dynamics

The role of the dynamics is to supply a differentiable state derivative function $\dot{\vec{x}} = f(\vec{x}, \vec{u})$ and a corresponding discrete update map. The implementation is built in CasADi so that first- and second-order derivatives are available to the NLP solver without finite differencing.

Two design goals guided the implementation. First, the model must remain physically interpretable, so we keep a rigid-body structure with explicit aerodynamic forces and moments. Second, the model must remain numerically robust inside trajectory optimization, so state integration, quaternion handling, and time parameterization are exposed as configurable options in the control layer.

In practice this leads to a modular structure: the rigid-body equations define the state derivative, the aerodynamic block supplies coefficients, and the control problem chooses integration and normalization constraints. This separation allowed rapid experimentation with model fidelity while keeping the optimization interface unchanged.

2.1 6 Degree of Freedom Rigid Body

: We model the aircraft as a rigid body, subject to forces and torques due to the aerodynamics and gravity. Since we want to perform motion planning it is natural to define the position and velocity in the iner-

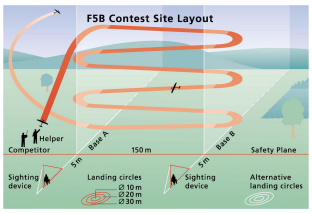


Figure 2: Example trajectory exhibiting gimbal lock. Note the singularity in the angular rates (red dotted line) and the switch between pitch and yaw angles.

tial frame, performing frame conversions to calculate the aerodynamic forces and moments.

We use the quaternion representation of the orientation instead of the Euler angles to avoid gimbal lock (generated when two of the defining axes for the Euler angles become parallel). As can be seen in Fig. 2 we often see such effects when flying the aggressive maneuvers required for optimality.

2.1.1 State Space $\mathbb{V} = \mathbb{R}^3 \times \mathbb{R}^3 \times SO(3) \times \mathbb{R}^3$

Inertial Position $\vec{p} = (x, y, z)^T \in \mathbb{R}^3$

Inertial Velocity $\vec{v} = (u, v, w)^T \in \mathbb{R}^3$

Orientation $\underline{q} = (q_1, q_2, q_3, q_0)^T \in SU(2)$

Angular Velocity $\vec{\omega} = (p, q, r) \in \mathbb{R}^3$

2.1.2 Reference Frames

We work primarily in two reference frames, the inertial and the body frame. Rather than defining a separate wind-frame state, wind effects are introduced as relative velocity in the aerodynamic calculations. Conversion between inertial and body vectors is facilitated by the quaternion sandwich product:

$$\text{rotate}(\underline{q}_I^B, \vec{v}_I) \equiv \underline{q}_B^I \otimes (0, \vec{v}_I) \otimes \underline{q}_I^B, \tag{1}$$

where $\underline{q}_B^I = (\underline{q}_I^B)^{-1}$.

2.1.3 Forces and Moments

eqn. for xdot in inertial frame as well as frame conversions. explain where gravity comes in and why we calculate forces in the body and not in the inertial frame.

2.2 Aerodynamics

2.2.1 Aerodynamic State

The aerodynamic state of the aircraft is fully represented by the aerodynamic state variables:

Dynamic Pressure

$$\bar{q} = \frac{\rho}{2} v_{rel}^2 \tag{2}$$

Angle of Attack

$$\alpha = \arctan\left(\frac{v_{rel,z}}{v_{rel,x}}\right) \tag{3}$$

Sideslip

$$\beta = \arcsin\left(\frac{v_{rel,y}}{v_{rel}^2}\right) \tag{4}$$

Control Variables

- Aileron Deflection: δa
- Elevator Deflection: δe
- Rudder Deflection: δr

Angular Rates $\vec{\omega} = (p, q, r)^T$

This is the input to our aerodynamics model.

2.2.2 Aerodynamics Model

We use the conventional approach to modeling aircraft dynamics using aerodynamic coefficients. We define the lift L as:

$$L = \bar{q} S C_L, \tag{5}$$

where S is the reference area (a measure of the exposed lifting surface) and C_L is the lift coefficient. This definition extracts the velocity dependence from the coefficient and places it in $\bar{q}$. One can think of $\bar{q}S$ as a measure of the kinetic energy being deposited

into the lifting surface. Due to our reference frame convention we deviate slightly from the normal definitions, defining instead:

$$F_X = \bar{q}SC_X \tag{6}$$
$$F_Y = \bar{q}SC_Y \tag{7}$$
$$F_Z = \bar{q}SC_Z \tag{8}$$
$$M_l = \bar{q}SbC_l \tag{9}$$
$$M_m = \bar{q}ScC_m \tag{10}$$
$$M_n = \bar{q}SbC_n \tag{11}$$

where the torque components M_l, M_m, M_n are additionally multiplied by their moment arms span b and aerodynamic chord c.

The implementation follows this coefficient-based structure exactly and separates "how coefficients are represented" from "how forces and moments are applied". Given the aerodynamic input vector $[\bar{q}, \alpha, \beta, \delta_a, \delta_e]^T$, the model outputs $[C_X, C_Y, C_Z, C_l, C_m, C_n]^T$. The same downstream force and moment equations are then used for all model types.

To support different data availability regimes, four interchangeable coefficient models are implemented:

- Default analytical model (low-parameter baseline with linear damping terms),
- Linear regression model (matrix map from features to coefficients),
- Polynomial model (CasADi-compatible fitted functions),
- Neural model (L4CasADi wrapper for learned surrogate inference).

This abstraction was a deliberate design decision. It enables ablations between physically constrained and data-driven models while keeping the rest of the simulation and optimization stack identical. In addition, optional stall scaling is implemented through smooth sigmoid factors, which avoids hard discontinuities and preserves solver-friendly derivatives near the edge of the envelope.

2.3 System Identification

To generate our dynamics model we need to capture the aerodynamics of the aircraft. We explore two routes of data acquisition and explain their implementations into the control pipeline.

2.3.1 System Identification from CFD Simulation

In principle, CFD-generated datasets are well suited to this framework because the dynamics interface only requires samples of $(\bar{q}, \alpha, \beta, \delta_a, \delta_e) \mapsto (C_X, C_Y, C_Z, C_l, C_m, C_n)$. Once these samples are available, the same fitting pipeline can be used to identify either linear, polynomial, or neural coefficient models.

The key implementation decision was therefore to keep model fitting and model use decoupled: the control solver consumes a single coefficient function regardless of how it was identified.

2.3.2 System Identification from In-Flight data

In-flight identification is handled by the same abstraction layer. Recorded trajectories are transformed into aerodynamic state variables, and the coefficient models are fitted offline. The resulting artefact (for example, a linear coefficient matrix, a polynomial fit, or a neural network checkpoint) is then loaded by the dynamics class through a model registry.

This design was chosen because it enables rapid iteration: new datasets or regressors can be tested without changing either the rigid-body equations or the control formulation.

2.4 Damping Terms

Without access to dynamic windtunnel tests or CFD simulations we cannot directly model the dynamic aerodynamic effects that serve to damp the roll, pitch and yaw rates. We propose that these dynamic effects can be approximated by calculating the effective angle of attack experienced by the lifting surfaces under the effects of the angular rates. We consider

only the dominant terms, eliminating all off-diagonal terms apart from the yaw-roll coupling and make the following simplifying assumptions:

1. For each lifting surface the lift acts at its centre of pressure.
2. The lift experienced is the same as the static lift for the inferred aerodynamic state.
3. The dominant dynamic effects are captured by modifying local inflow, rather than introducing a full unsteady aerodynamic state.

2.4.1 Example

For a plane yawing with yaw rate r (anti-clockwise about the downward z-axis) the right wing is travelling backwards and the left is travelling forwards. This results in a modification to the dynamic pressure $\bar{q}$ of each wing according to

$$\bar{q}_{l/r} = \frac{1}{2}\rho(\vec{v}_{rel} \pm \frac{b}{4} r\hat{x}), \tag{12}$$

where $\hat{x}$ is the unit vector in the body-frame forward direction.

Similarly, under yaw rate r, the rudder experiences a change in relative airspeed:

$$\vec{v}_{rel,rudder} = \vec{v}_{rel} - d_{rudder} r, \tag{13}$$

where d_{rudder} is the moment arm of the rudder. This modifies the effective angle of sideslip for the lifting surface in accordance with equation 4.

In the implementation, this idea is encoded through effective local aerodynamic variables:

$$\alpha_{elev} = \arctan 2(w + d_{rudder} q,\ u + \epsilon), \tag{14}$$

$$\alpha_L = \arctan 2(w - \tfrac{b}{4} p,\ u + \epsilon), \tag{15}$$

$$\alpha_R = \arctan 2(w + \tfrac{b}{4} p,\ u + \epsilon), \tag{16}$$

and an effective rudder sideslip constructed from the yaw-rate-corrected lateral velocity.

This approximation was selected as a compromise between fidelity and optimization tractability. It captures the dominant damping couplings (p, q, r effects on aerodynamic loads) while preserving a compact state and smooth symbolic derivatives.

2.5 Integration and Simulation

2.5.1 Integration Methods

The dynamics class exposes a differentiable state derivative $\dot{\vec{x}} = f(\vec{x}, \vec{u})$ and a discrete update $\vec{x}_{k+1} = F(\vec{x}_k, \vec{u}_k, \Delta t_k)$. In the current implementation, F is primarily constructed using RK4. For numerical stiffness or formulation experiments, the control layer also supports an implicit defect form based on f.

Time can be parameterized in four modes:

- fixed step,
- progress-based step,
- variable step,
- adaptive step with an error surrogate constraint.

The motivation was to compare solver sparsity and conditioning trade-offs without changing the physical model.

2.5.2 Quaternions

Orientation is propagated with quaternion kinematics to avoid Euler-angle singularities. To maintain unit norm, three alternatives are supported in optimization:

- direct normalization after integration,
- hard algebraic constraint $\|q\|^2 = 1$,
- Baumgarte stabilization on $\phi = \|q\|^2 - 1$.

This configurability was added because each method has different effects on NLP conditioning and constraint activity.

2.5.3 Numerical Stability

The implementation prioritizes smooth symbolic expressions and bounded decision variables to improve solver robustness. In particular, progress/time variables are constrained positive, and optional adaptive stepping penalizes large local truncation surrogates. Together with quaternion stabilization, this reduces drift and improves repeatability across aggressive maneuvers.

2.6 Constraints on the Flight Envelope

2.6.1 Angle of Attack

To keep the optimizer in a physically plausible region, we constrain angle of attack as

$$-20^\circ \leq \alpha \leq 20^\circ. \tag{17}$$

This is intentionally tighter than a purely simulation-only envelope, because optimization otherwise tends to exploit high-α states with poor model validity.

2.6.2 Angle of Sideslip

Sideslip is constrained to

$$-10^\circ \leq \beta \leq 10^\circ, \tag{18}$$

which limits lateral-flow conditions where simple coefficient models are less reliable and where aggressive rudder activity can destabilize the solve.

2.6.3 Airspeed

The body-relative airspeed is bounded as

$$20\,\mathrm{m/s} \leq V \leq 100\,\mathrm{m/s}, \tag{19}$$

implemented via a squared-speed bound in the NLP.

In addition, control surfaces are bounded in the control layer ($\delta_a, \delta_e, \delta_r \in [-5^\circ, 5^\circ]$), and altitude is kept below the NED reference plane ($z < 0$). Together, these constraints define the feasible envelope used for all reported optimization experiments.

3 Control

Control seeks to solve the problem:

$$\vec{u}^* = \underset{\vec{u}}{\operatorname{argmin}} J(\vec{x}, \vec{u}, t), \tag{20}$$

subject to constraints on the state and control variables:

$$\vec{g}(\vec{x}, \vec{u}) \geq \vec{0}, \tag{21}$$

$$\vec{h}(\vec{x}, \vec{u}) = \vec{0}, \tag{22}$$

where $J : X \times U \times T \to \mathbb{R}$ is a scalar objective function that we seek to minimise via optimal choice of control inputs $\vec{u}^*$.

In implementation, this problem is solved with a direct transcription approach using CasADi Opti. The horizon is discretized into control nodes, and the solver optimizes states, controls, and optionally time/progress variables simultaneously. Dynamics are enforced through defect constraints between adjacent nodes.

This formulation was chosen for three reasons:

- it keeps the full nonlinear dynamics in the loop,
- it exposes sparse Jacobian/Hessian structure to IPOPT,
- it allows us to switch objectives (goal, min-time, progress tracking, waypoint traversal) without rewriting the core constraints.

Two implementation choices are central. First, state and control envelopes are enforced directly at every node. Second, quaternion consistency is treated as a configurable numerical policy (normalization, hard constraint, or Baumgarte stabilization), allowing us to trade off strict feasibility and solver conditioning.

3.1 Goal Acquisition

Goal acquisition is implemented as a terminal objective on position, with additional terms to discourage overly aggressive actuation and unstable terminal velocity. In its basic form, the optimization includes

$$J_{goal} = w_g \left\| \vec{p}_N - \vec{p}_{goal} \right\|^2, \tag{23}$$

combined with smoothing penalties on control variation $\Delta\vec{u}_k = \vec{u}_{k+1} - \vec{u}_k$.

The practical reason for this structure is that a pure terminal distance loss often produces unrealistic bang-bang control when the aircraft is close to the target. Penalizing control increments makes the

resulting maneuvers easier to track, and adding terminal velocity shaping reduces overshoot at the end of the planning horizon.

Within the same formulation, state and envelope constraints (airspeed, angles, altitude) remain active, so goal convergence is always sought inside the feasible flight domain rather than in an unconstrained kinematic space.

3.2 Minimum Time

To handle minimum time planning we must

introduce time as an optimization variable while preserving numerical structure. Rather than optimizing a single terminal time only, the implementation uses per-node progress/time variables to define local step sizes.

For each segment, a positive progress variable is introduced and mapped to local time through a smooth transform. This guarantees $\Delta t_k > 0$ by construction and allows bounds on local step duration. The minimum-time objective is then formed from the sum of local durations, which in code is implemented through smooth expressions of the progress rates.

This design was selected to keep the NLP well-scaled: directly optimizing unconstrained Δt_k can produce poor conditioning, whereas bounded progress variables provide better numerical behavior and make it straightforward to compare fixed, variable, and adaptive time modes.

The same mechanism also enables adaptive timestepping by adding a local integration-error surrogate constraint, so temporal resolution increases only where dynamics become difficult.

3.3 Progress Maximisation

3.3.1 Dubins Path

Progress maximization is built around a geometric reference path generated from Dubins-style primitives. The Dubins initializer provides a feasible curve with curvature-aware heading transitions, which is then used as a tracking manifold for optimization. This gives a reliable initial guess and reduces the chance of solver failure in long horizons.

3.3.2 Progress Variables

In the moving-horizon tracking controller, each node carries a scalar progress state $s_k \in [0, 1]$ along the reference path. A progress-rate surrogate is computed from velocity projected onto the local path tangent, and constraints enforce consistent forward evolution.

The objective combines:

- tracking error penalty to the path,
- reward on accumulated progress,
- reward on forward progress rate,
- penalties on backward motion and excessive control effort,
- terminal alignment with the end of the reference path.

This formulation was chosen instead of pure waypoint distance minimization because it provides a smoother optimization landscape and naturally handles trade-offs between speed and path adherence during aggressive maneuvers.

3.4 Waypoint Traversal

Waypoint traversal is implemented with a complementarity-based formulation, following the idea that the optimizer should decide when each waypoint becomes active while still enforcing ordered passage.

For each waypoint j and node k, auxiliary variables $\lambda_{j,k}, \mu_{j,k}, \nu_{j,k}$ are introduced. The constraints include:

$$\lambda_{j,k+1} - \lambda_{j,k} + \mu_{j,k} = 0, \tag{24}$$

$$\mu_{j,k} \geq 0, \tag{25}$$

$$\lambda_{j,k} - \lambda_{j+1,k} \leq 0, \tag{26}$$

and the complementarity condition

$$\mu_{j,k} \left(\|\vec{p}_k - \vec{w}_j\|^2 - \nu_{j,k} \right) = 0, \quad 0 \leq \nu_{j,k} \leq \tau^2. \tag{27}$$

The main design decision here was to avoid hard switching logic. Discrete waypoint-index updates are

difficult for smooth NLP solvers, whereas this continuous complementarity formulation preserves differentiability and lets the optimizer determine a consistent waypoint activation schedule.

4 Shape Optimality

5 Notation

5.1 Reference Frames

5.2 Trim

5.3 Stability

We examine the stability of the system by looking at the eigenvalues of the dynamics Jacobian for the continuous case and also the state update Jacobian in the discrete case. For the

5.4 Discrete Control

We discretise time and perform the state update with RK4

6 Theory

6.1 Flight Model:

We make use of the 6DOF flight model as described in [**aircraftflightdynamics**] as our dynamics depend on orientation as well as position. This gives us the dynamics:

$$x = 1 \tag{28}$$

6.2 Shortest Path Planning

The problem of minimum time planning can be approached in a number of ways. Common approaches include equating it to a corresponding shortest path problem, where the solutions can be described as successive arcs of limited curvature [3].

6.3 MPC

Other approaches stem from the field of optimal control. Model predictive control (MPC) seeks to find the control inputs that lead to a trajectory that minimises a given cost function. The control inputs that we seek are functions $\vec{u}(t)$, approaches for determining them can be broken into two categories: Direct (Discretise and Optimise) and Indirect (Optimise and Discretise).

6.3.1 Direct Methods

Here we discretise the optimisation problem, this gives us a constrained parameter optimisation problem that we then solve. Existing direct methods include:

- **Direct Collocation:** We approximate the trajectories $(\vec{x}(t), \vec{u}(t))$ by polynomial splines.
- **Orthogonal Collocation:** As above but the trajectories are represented by orthogonal, high-order splines.
- **Spectral Collocation:** Here the important aspect is that the collocation nodes are chosen to be the roots of orthogonal basis polynomials that characterise the problem. These basis polynomials are chosen based on a discretised form of the dynamics.
- **Multiple Shooting:** Here we discretise the trajectory and add a constraint between each segment. The resulting large sparse non-linear program can then be solved numerically.

6.3.2 Indirect Methods

Indirect methods first consider the system dynamics to construct necessary and sufficient conditions for optimality. These conditions are then discretised and solved. Examples of indirect methods include:

- **Temporal Finite Elements:**
- **Single Shooting:** We pick a set of control inputs and simulate the trajectory to completion, repeating the approach until we have sufficiently minimised the objective.

7 Dynamics?

The goal of the dynamics model is to supply the state update information:

$$\dot{\vec{x}}(t) = f(\vec{x}(t), \vec{u}(t)) \tag{29}$$

Which means that we want a function:

$$f_{dyn} : \mathbb{R}^d \times \mathbb{R}^d_u \times \mathbb{R} \to \mathbb{R}^d \tag{30}$$

$$f_{dyn}(\vec{x}, \vec{u}, t) = \frac{d\vec{x}}{dt} \tag{31}$$

So that a state update can be computed as:

$$\vec{x}_{t+1} = \vec{x}_t + f_{dyn}(\vec{x}(t), \vec{u}(t), t)\delta t \tag{32}$$

7.1 Approaches to Modelling Dynamics

7.1.1 The Naive Approach:

We can use a data driven approach to naively fit f_{dyn}. This is what is done in [**scaramuzza1**] but is very data intensive.

7.1.2 Physics Informed:

Conventional Lookup tables for $C_{L,\alpha}$ etc.

8 Comparing Toolboxes

This section is an altered excerpt from [1] where all optimisation tools wholly unsuitable (due to difficulty of implementing the task, lacking interoperability with the other components of the system or missing features).

There are many software packages and modeling languages currently available for optimization and optimal control. This section, while not a comprehensive comparison, attempts to summarize some of the distinguishing features of each package.

Pyomo [19] is a Python package for modeling and optimization. It supports automatic differentiation and discretization of DAE systems using orthogonal collocation or finite-differencing. The resulting nonlinear programming (NLP) problem can be solved using any of several dozen AMPL Solver Library (ASL) supported solvers.

JuMP [20] is a modeling language for optimization in the Julia language. It supports solution of linear, nonlinear, and mixed-integer problems through a variety of solvers. Automatic differentiation is supplied, but, as of writing, JuMP does not include built-in support for differential equations.

Casadi [21] is a framework that provides a symbolic modeling language and efficient automatic differentiation. It is not a dynamic optimization package itself, but it does provides building blocks for solving dynamic optimization problems and interfacing with various solvers. Interfaces are available in MATLAB, Python, and C++.

AMPL [23] is a modeling system that integrates a modeling language, a command language, and a scripting language. It incorporates a large and extensible solver library, as well as fast automatic differentiation. AMPL is not designed to handle differential equations. Interfaces are available in C++, C#, Java, MATLAB, and Python.

JModelica [25] is an open-source modeling and optimization package based on the Modelica modeling language. The platform brings together a number of open-source packages, providing ODE integration through Sundials, automatic differentiation through Casadi, and NLP solutions through IPOPT. Dynamic systems are discretized using both local and pseudospectral collocation methods. The platform is accessed through a Python interface.

In addition to those listed above, many other software libraries are available for modeling and optimization, including AIMMS [31], CVX [32], CVXOPT [33], YALMIP [34], PuLP [35], POAMS, OpenOpt, NLPy, and PyIpopt.